Lab 1: Coordinate Frames

Introduction to Robotics DT512A

Temur Mohammad Nasr, Nicolaus Hentschel

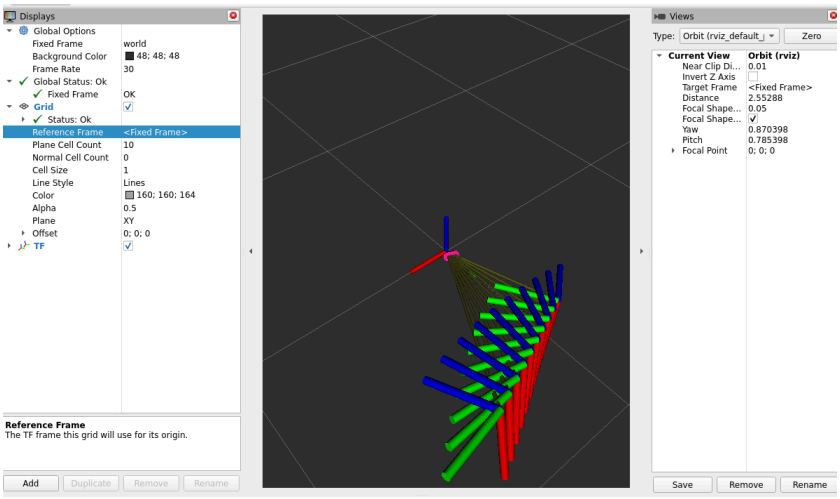
Task 2: Shortest Transform

Our node shortest_transform does:

- Create two random frames as Eigen::Affine3d: 3 Euler angles in [-pi,pi] (yaw-pitch-roll, z-y-x) + 3 translation components in [0,1]
- Computes the transform that brings frame 1 to frame : T1 * T12 = T2 -> T12 = $T1^{-1} \cdot T2$, then splits it into R & t
- Converts R to angle axis with eigen::AngleAxisd -> axis v and angle a. Eigen gives a in [0, pi], so the rotation is never more than half a turn -> is the shortest way to rotate
- Samples 10 steps with s = 1/9 for i = 0..9 -> a_i = s*a and t_i = s·t. Both ends are included. So step_0 = frame 1 and step_9 = frame 2
- Builds each step as T_i = T1*Translation(t_i) · AngleAxis(a_i, v)
- Converts every transform to a TF message via a quaternion (Eigen::Quaterniond) and publishes frame1 and frame2 and step_0…step_0 relative to world with a 10 Hz timer.

Example run:

- t= [0.116, -0.597, 0.149], |t| = 0.627 m
- v= [0.972, -0.163, 0.169], a = 1.187 rad (68.0°)
- Last step vs frame2: position error = 1.57e-16, rotation error = 8.80e-16 -> basically 0, only rounding.



Rviz with Fixed Frame = world and a TF display. The frames turn step by step from frame1 to frame2.

Observations:

- All steps rotate around the same axis v with the same angle per step -> rotation is spread out evenly
- The translation is applied before the rotation
- Every run gives new random frames so the numbers above change each time

Task 3: Forward Kinematics with KDL

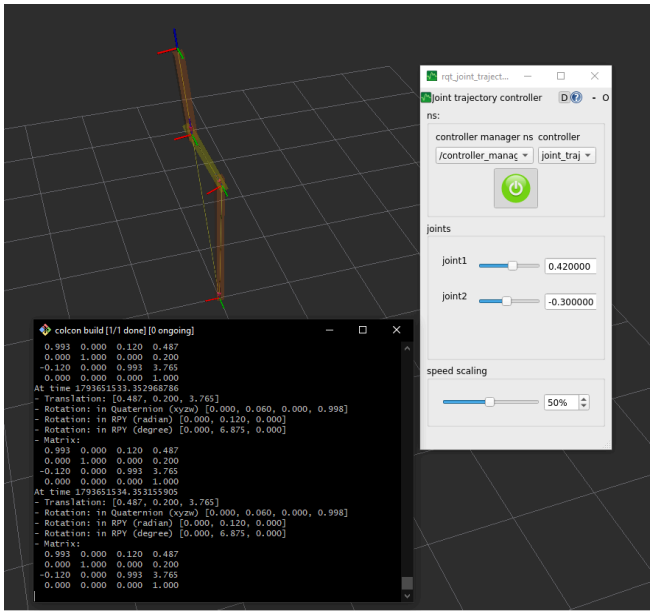
Our node test_fk does the following:

- Reads the parameter urdfPath and reads the file into a string
- kdl_parser::treeFromString -> KDL::Tree and then getChain(base_link, tool_link) -> a chain with 2 joints
- Publishes a TF transform base_link -> my_fk, identity at the start with a 10 ms timer -> 100Hz
- Subscribes to joint_states. In the callback we match the joint values by name -> ChainFkSolverPos_recursive::JntToCard -> pose of tool_link -> position + quaternion go into the transform we publish

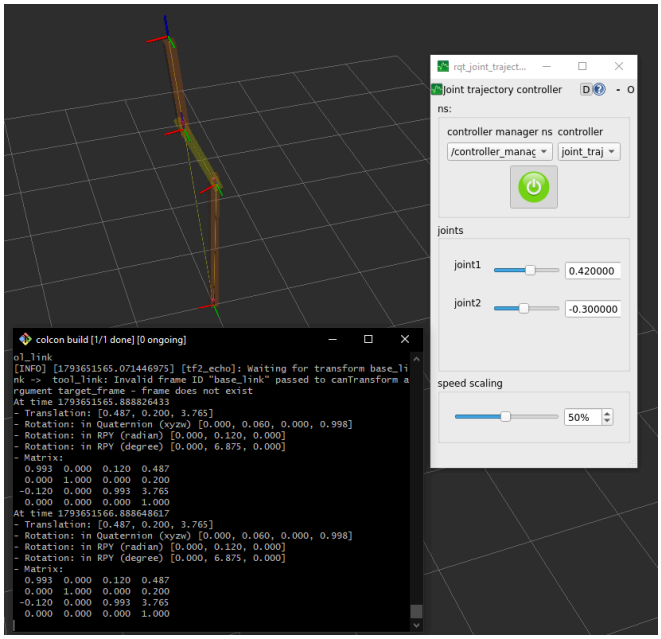
Launch file test_fk.launch.py:

- Includes rrbot.launch.py
- Runs xacro to turn rrbot.urdf.xacro into a plain URDF in /tmp/rrbot.urdf. Kdl_parser can't read xacro syntax
- Starts test_fk only after xacro finished, otherwise the file might not exist yet.

Verification with both joints bent, q1 = 0.42 and q2 = -0.30. We compared our frame with the one robot_state_publisher computes from the URDF:
Tf2_echo base_link my_fk



Tf2_echo base_link tool_link (robot_state_publisher)



Both give translation = [0.487, 0.200, 3.765] and rotation = 0.120 rad (6.875°) about y -> identical

In RViz the my_fk frame sits exactly on the tip of the arm.

Additional codes in the .cpp files. Both nodes are in our package lab01.